

Project 3

Voice

Recognition Model

Use Case : MINDS14



Berikut adalah **gambaran singkat dari project yang akan dilakukan**:



Project Description

Proyek ini berfokus pada pengembangan sistem **Voice Recognition (Automatic Speech Recognition/ASR)** yang mampu mengubah sinyal audio menjadi teks. Sistem dirancang untuk memahami berbagai variasi gaya bicara, aksen, dan dialek guna meningkatkan interaksi manusia dengan komputer.



Project Method

Pendekatan yang digunakan dalam proyek ini meliputi:

- Pemanfaatan **pretrained model** (seperti Whisper atau Wav2Vec2)
- **Fine-tuning model** menggunakan dataset **MInDS-14**
- Preprocessing audio dan ekstraksi fitur
- Training dan evaluasi model pada data uji

Metode ini bertujuan untuk menghasilkan model yang akurat dalam mengenali ucapan.



Project Output

Output utama dari proyek ini adalah:

- ✓ Model **speech-to-text (ASR)** yang telah dilatih
- ✓ Hasil evaluasi performa model
- ✓ Insight dan analisis hasil eksperimen
- ✓ Repository proyek yang dipublikasikan di **GitHub**



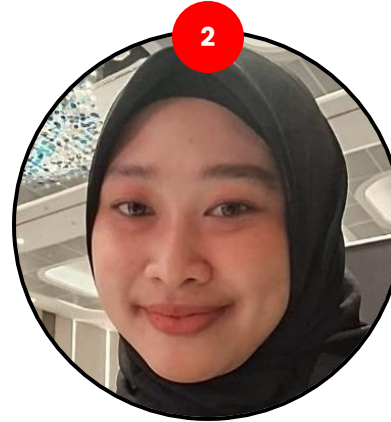
Business Objective

Tujuan bisnis dari proyek ini adalah:

- ❖ **Meningkatkan efisiensi** layanan berbasis suara (customer service, chatbot, dll.)
- ❖ **Mendukung otomatisasi transkripsi** (rapat, wawancara, dll.)
- ❖ **Meningkatkan pengalaman pengguna** dalam interaksi berbasis suara
- ❖ **Menjadi dasar pengembangan produk** berbasis AI suara di berbagai industri



Irfan Gani Alim



Khaerani **Arista** Dewi



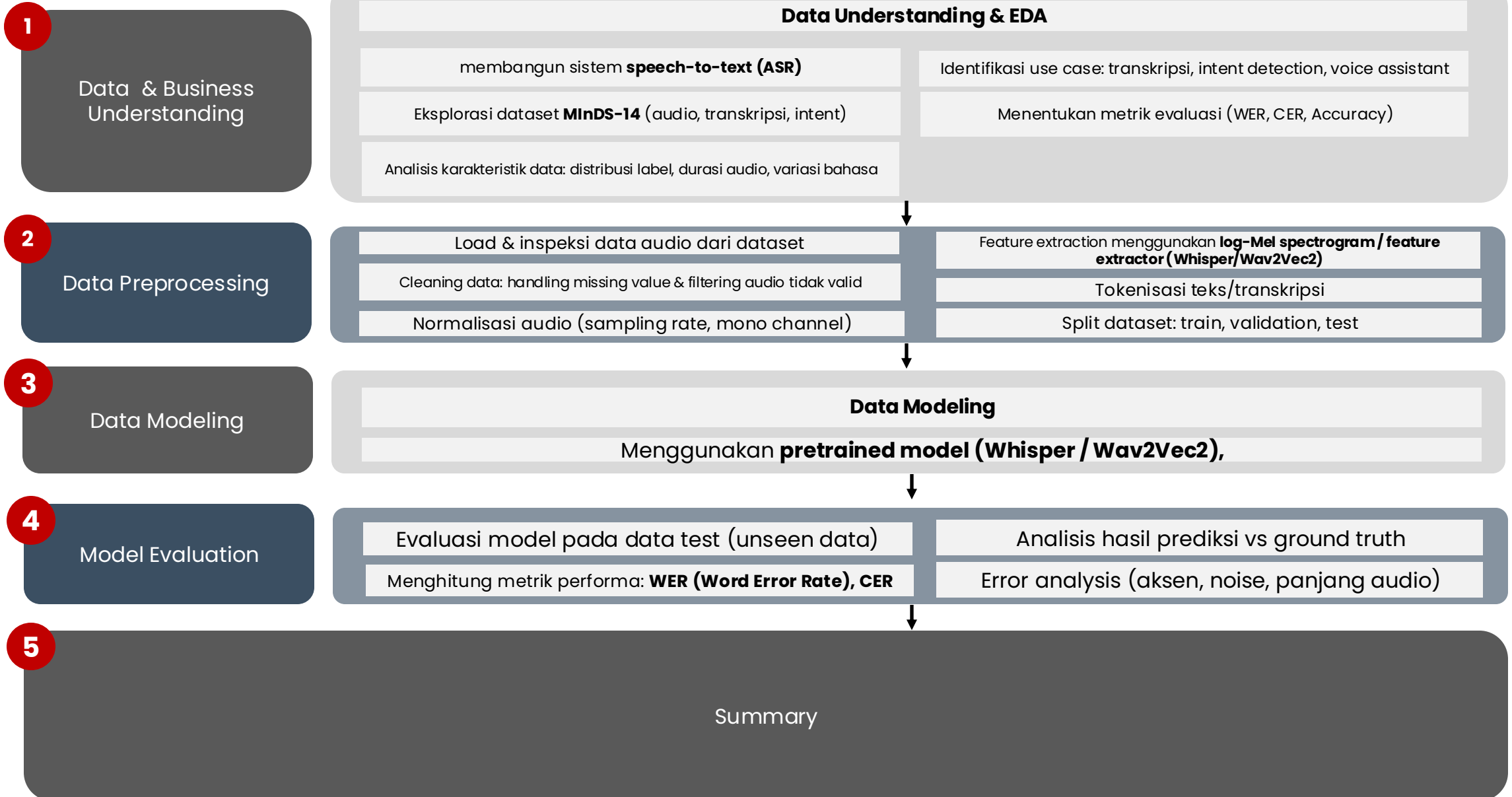
Agil Setiawan



Krisna **Fery** Rahmantya



Surya Dharma Putra



Install & Import Libraries 1

- Library yang dibutuhkan:
- **pandas, numpy:** untuk manipulasi data
 - **pyarrow:** untuk engine baca file parquet
 - **matplotlib, seaborn:** untuk visualisasi
 - **librosa:** analisis audio (waveform, spectrogram, MFCC)
 - **soundfile:** decode audio bytes ke array numpy
 - **IPython.display.Audio:** play audio di notebook

Load & Mapping 2

Membuat kolom baru `intent\_name` di awal, semua visualisasi, analisis, dan grouping yang dilakukan di step-step berikutnya bisa langsung memakai label yang mudah dibaca manusia

Result:

Shape: (563, 7)
Missing: 0
Kolom: ['path', 'audio', 'transcription', 'english_transcription', 'intent_class', 'lang_id', 'intent_name']

Drop Redundant Column 3

Dari tujuh kolom yang ada, dua di antaranya akan di-drop **karena redundant:**

- **english_transcription:** isinya identik dengan kolom 'transcription' karena dataset ini hanya berisi split en-US.
- **lang_id:** nilainya konstan untuk semua 563 baris (karena hanya ada satu Bahasa)

Result:

Kolom sekarang:
['path', 'audio', 'transcription', 'intent_class', 'intent_name']
Shape: (563, 5)

Helper Decode Audio 4

- Menggunakan **fungsi helper decode_audio()** untuk mengkonversi bytes audio menjadi array numpy yang siap diproses.

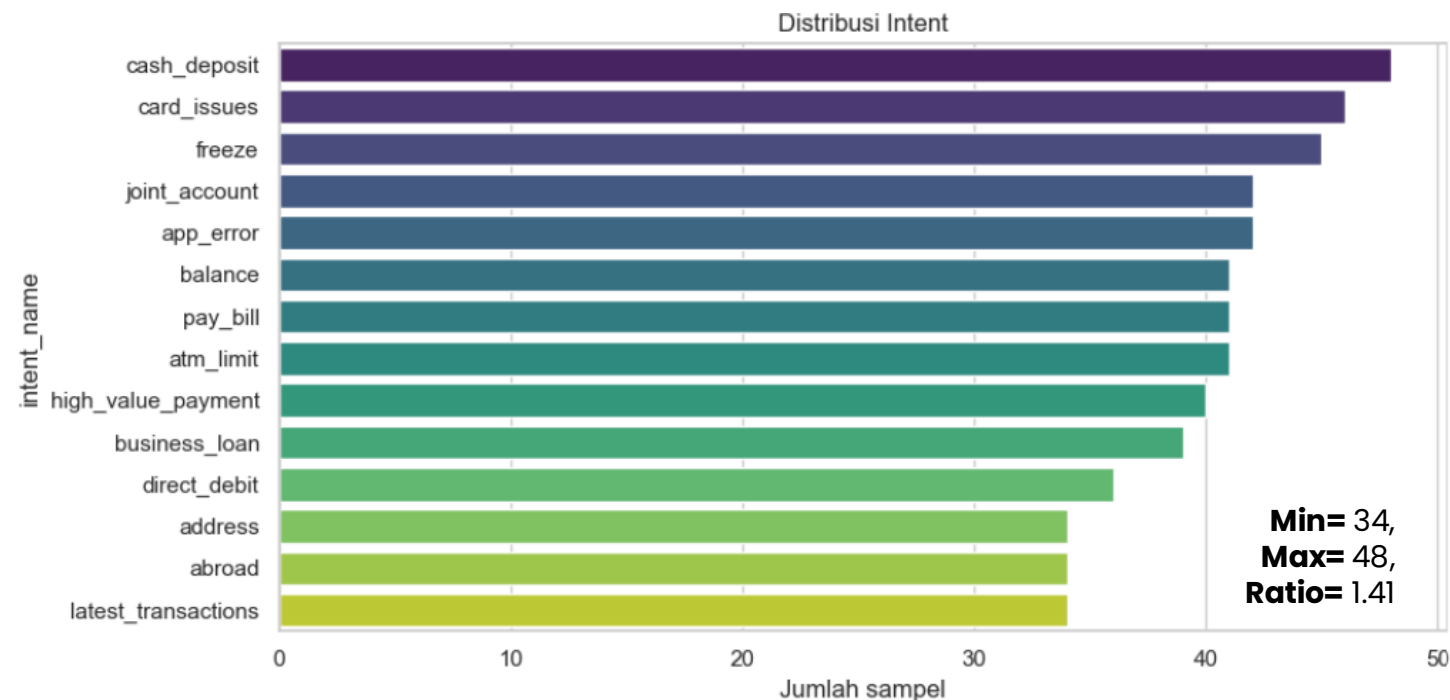
Result:

sr= 8000,
shape= (86699,),
durasi= 10.84s

5

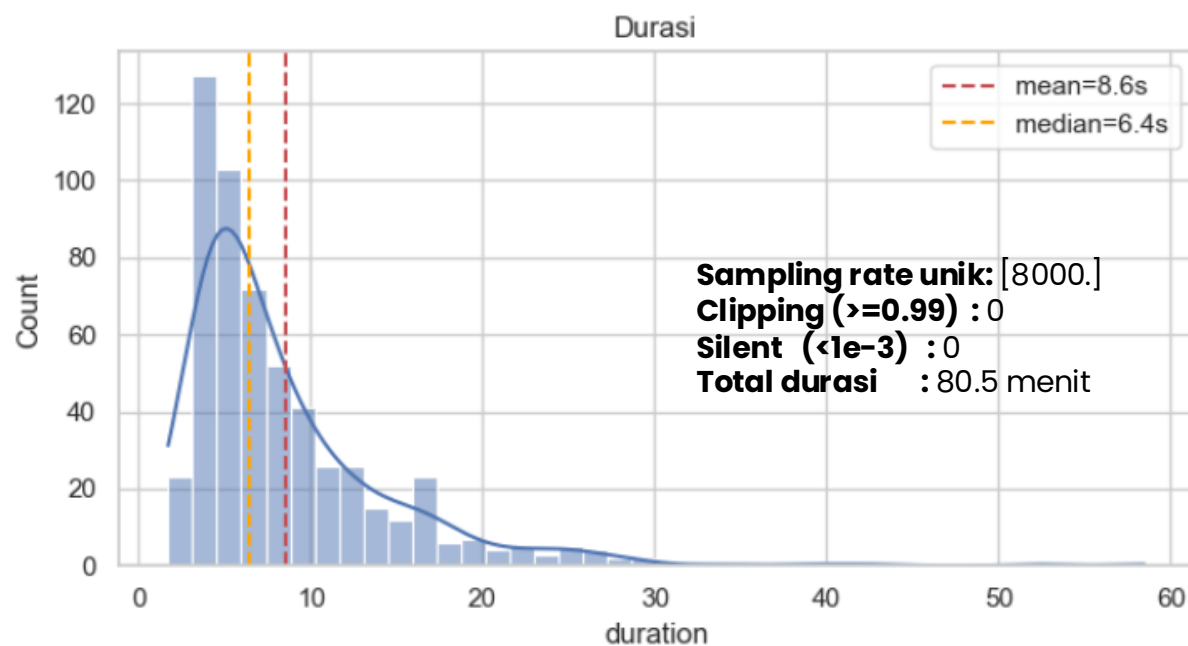
Distribusi Intent

- Visualisasi distribusi intent menunjukkan **dataset cukup seimbang di seluruh 14 kelas**.
- Intent terbanyak adalah **cash_deposit** (48 sampel),
- Intent **paling sedikit** adalah **address, abroad, dan latest_transactions** (masing-masing 34 sampel).
- Rasio **max/min** hanya **1,41 tergolong ringan** dan tidak memerlukan class weighting atau oversamplin.
- Dengan **total 563 sampel dibagi 14 intent** (rata-rata ~40 sampel per intent), ukuran dataset ini kecil untuk deep learning.
- **Strategi yang dipilih adalah transfer learning** dengan **fine-tuning dua model** pre-trained (wav2vec2 dan Whisper) untuk dibandingkan performanya.
- Untuk **split data**, memakai **stratified split** agar setiap intent terwakili proporsional di train/val/test, terutama untuk kelas minoritas.

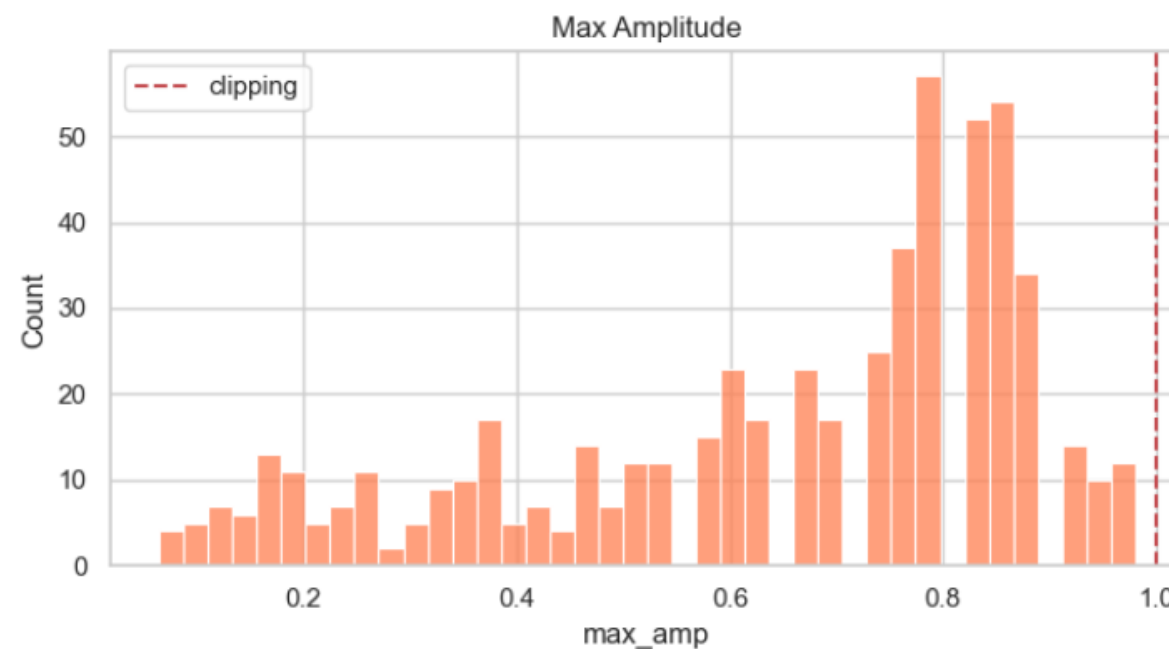


Metadata Audio (Durasi & Amplitude)

Sampling rate seragam di 8.000 Hz untuk semua file, khas kualitas audio telepon. **Karena wav2vec2 dan Whisper pre-trained di 16 kHz, langkah resampling menjadi wajib sebelum fine-tuning.**



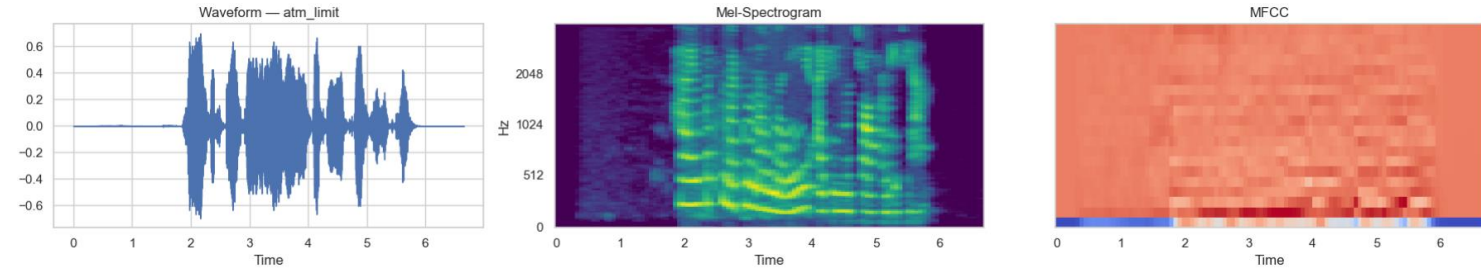
Histogram durasi menunjukkan distribusi yang jelas **right-skewed** dengan **puncak di rentang 3–6 detik**. Garis median (6,4s) di kiri mean (8,6s) menegaskan bahwa mayoritas audio pendek, dengan beberapa outlier panjang hingga ~58 detik yang menarik rata-rata ke atas.



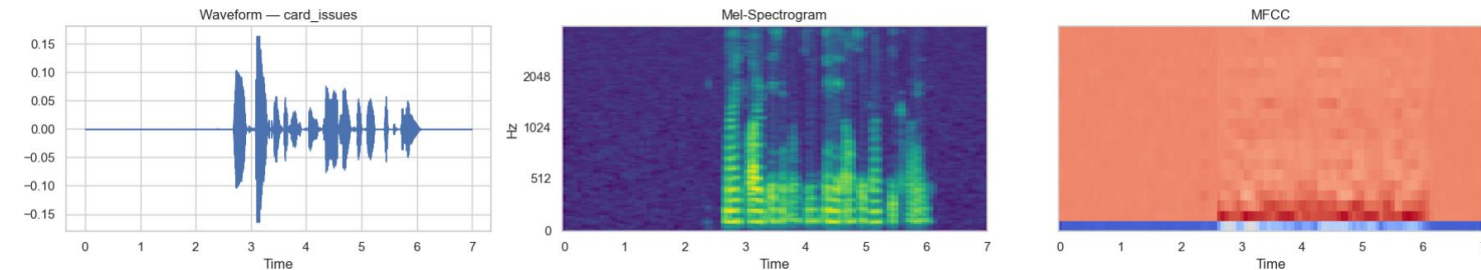
Histogram max amplitude memperlihatkan puncak dominan di rentang 0,75–0,90, jauh dari ambang clipping (garis merah di 1,0). **Kombinasi dengan hasil deteksi anomali, 0 clipping, 0 silent, mengkonfirmasi dataset benar-benar bersih dari distorsi maupun file kosong.**

Visualisasi Audio (Waveform, Spectrogram, MFCC)

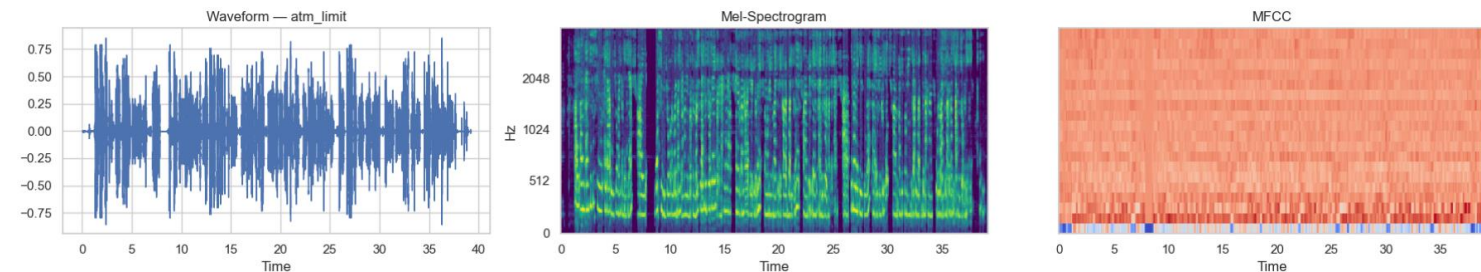
Pemutar audio pada ketiga sampel menghasilkan suara yang jernih dan jelas terdengar, dengan konten ucapan yang sesuai dengan label intent-nya. Kualitas audio telepon (8 kHz) memang terdengar lebih "tipis" dibanding audio HD, namun kata-kata tetap dapat dipahami dengan baik. Tidak ada distorsi, noise berlebih, atau artefak yang mengganggu, dataset siap untuk tahap modeling.



Sampel 1 (atm_limit, ~7 detik) memperlihatkan pola yang ideal: waveform menunjukkan silence di detik 0–1,8 diikuti ucapan aktif hingga detik 6. Pola ini terkonfirmasi di mel-spectrogram (energi muncul mulai detik 2 di frekuensi rendah <1.000 Hz) dan di MFCC (koefisien 0 berubah dari biru/silence ke merah/aktif di titik yang sama).



Sampel 2 (card_issues, ~7 detik) menunjukkan amplitudo yang jauh lebih kecil (range $\pm 0,15$ vs $\pm 0,6$ di sampel pertama), menandakan speaker berbicara lebih pelan atau jarak ke mikrofon lebih jauh. Meski begitu, formant tetap terlihat jelas di mel-spectrogram dan MFCC tetap mampu menangkap boundary silence-to-speech di detik 2,5.

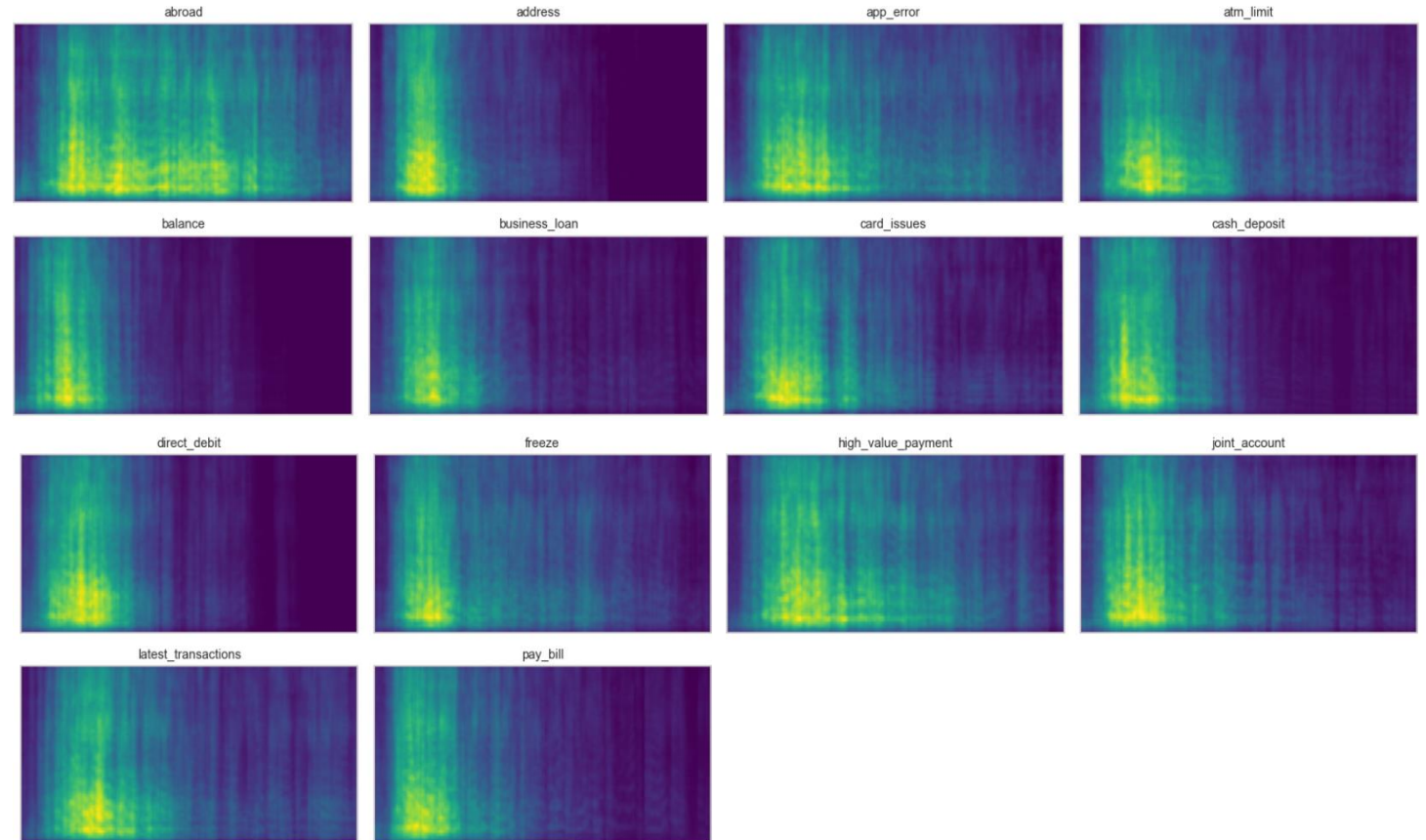


Sampel 3 (atm_limit, ~38 detik) adalah salah satu outlier durasi panjang. Waveform menunjukkan ucapan kontinu dengan beberapa jeda natural, mel-spectrogram menampilkan variasi fonem yang kaya sepanjang durasi, dan MFCC menunjukkan pola yang jauh lebih padat dibanding dua sampel pendek.

Rata – Rata Mel Spectrogram per Intent

8

Kemiripan visual antar intent menegaskan bahwa **klasifikasi ini tidak bisa diselesaikan hanya dengan fitur spektral permukaan**, model perlu memahami konten linguistik (kata dan fonem yang diucapkan), bukan sekadar pola energi frekuensi. **Ini menjadi alasan kuat kenapa strategi fine-tuning wav2vec2 dan Whisper tepat untuk kasus ini, karena kedua model sudah belajar representasi fonetik dan linguistik** yang mampu menangkap perbedaan semantik halus antar intent.



Visualisasi rata-rata mel-spectrogram dari 14 intent menunjukkan pola yang mirip secara global: semua kelas memiliki konsentrasi energi di frekuensi rendah (area kuning di bagian bawah), karakteristik khas ucapan manusia terlepas dari kontennya. **Perbedaan utama antar intent hanya terlihat pada panjang area aktif,** intent dengan ucapan pendek seperti *address* dan *balance* punya energi terkonsentrasi di awal, sementara intent yang lebih panjang seperti *high_value_payment* dan *joint_account* memiliki energi tersebar lebih lebar sepanjang waktu.

Temuan	Keputusan
Sampling Rate 8 kHz	Resample ke 16 kHz untuk wav2vec2 dan Whisper
Durasi Right-Skewed	Dynamic Padding per Batch untuk wav2vec2, Whisper otomatis pad ke 30s
Dataset Kecil (-1,34 Jam)	Transfer learning, bukan training from the scratch
Distribusi Intent Seimbang Tapi Kelas Minoritas Hanya 34 Sampel	Stratified Split by Intent untuk Balance Topik di Train/Val/Test
Audio Bersih (No Clipping/Silent)	Tidak perlu Filtering atau Denoising
Pola Spektral Mirip Antar Intent	Butuh model yang paham konten linguistic, wav2vec2 dan whisper cocok
Dataset Terbatas	Tambah audio augmentation (noise, time – stretch, pitch-shift) untuk meningkatkan generalization

Persiapan Data Audio & Teks untuk Model ASR (Dataset MInDS-14)

1 Drop Redundant Column

- english_transcription: identik dengan transcription (hanya split en-US)
- lang_id: konstan untuk semua 563 baris (satu bahasa)
- Kolom tersisa: path, audio, transcription, intent_class → Shape: (563, 5)

2 Normalisasi Transkripsi

- Teks transkripsi di-lowercase dan di-strip whitespace
- Konsistensi saat tokenisasi
- Evaluasi WER/CER tidak bias karena perbedaan case

3 Decode + Resample 8→16 kHz

- Audio bytes di-decode → array numpy via soundfile
- Resample 8→16 kHz via torchaudio.transforms.Resample
- Kolom audio mentah dibuang untuk hemat memori

4 Audio Augmentation

- 3 teknik: Gaussian noise, time stretching, pitch shifting
- Implementasi manual via numpy + librosa
- Diterapkan probabilistik $p=0.5$ tiap teknik

5 Stratified Train/Val/Test Split

- Pembagian 70 / 15 / 15
- Stratifikasi berdasarkan intent_class
- 394 train | 84 val | 85 test

6 Apply Augmentation ke Train Set

- Augmentation HANYA ke train set
- Copy augmented digabung dengan data asli
- Train set: 394 → 788 sampel | val/test tetap

7 Konversi ke HF DatasetDict

- DataFrame pandas → format DatasetDict
- Tiga partisi: train, val, test
- Simpan kolom waveform & transcription untuk Trainer API

8 Processor wav2vec2 & Whisper

- wav2vec2-base-960h: waveform→input_values, transkripsi uppercase→labels CTC
- whisper-base: waveform→log-mel (80,3000), transkripsi→labels BPE subword

9 Data Collator

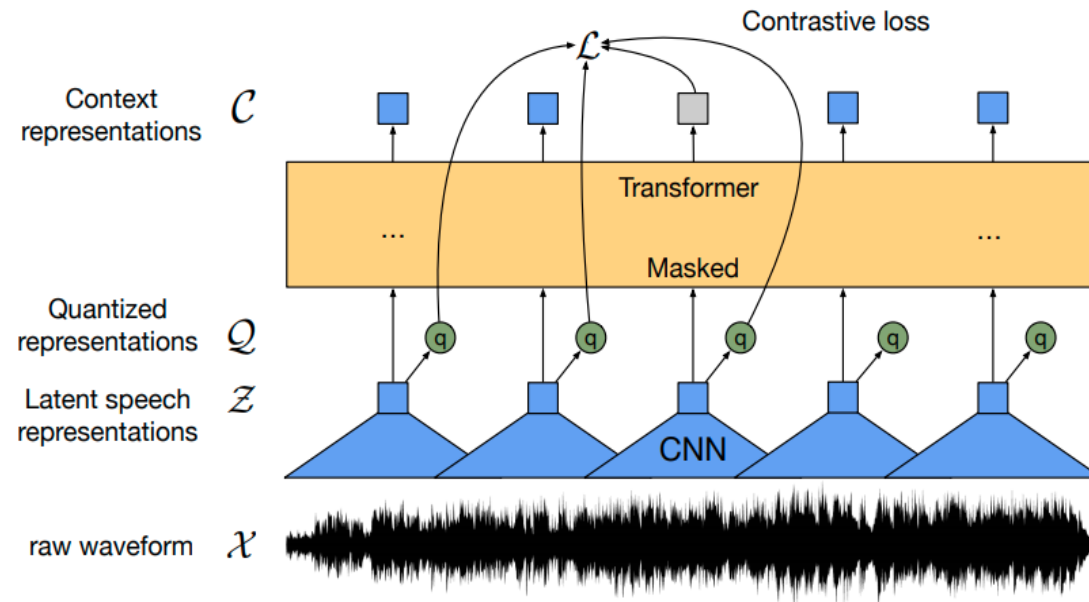
- DataCollatorCTC untuk wav2vec2: dynamic padding audio + label
- DataCollatorSeq2Seq untuk Whisper: pad label only, audio sudah fixed
- Token padding label = -100 → di-ignore saat hitung loss

Alasan di balik setiap pilihan teknis dalam pipeline pre-processing

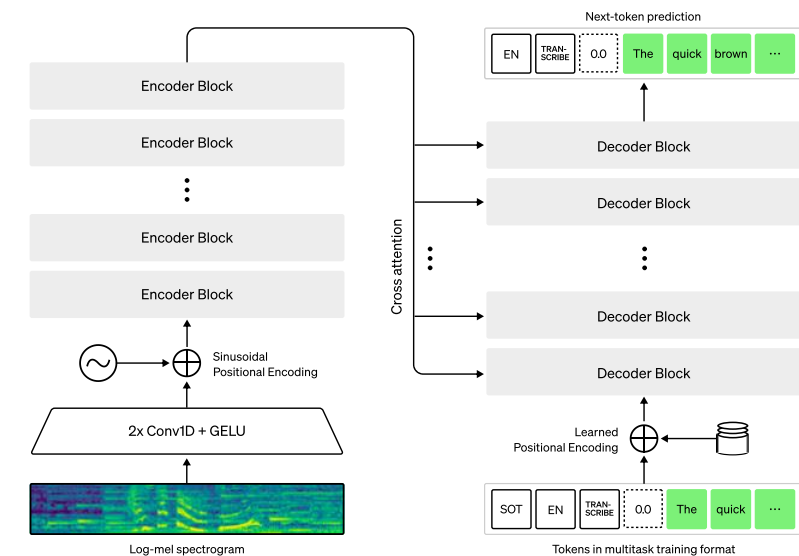
Keputusan	Alasan
Drop english_transcription & lang_id	Redundant: identik dengan transcription & nilai konstan untuk semua baris
Lowercase transkripsi	Konsistensi tokenisasi & evaluasi WER/CER
Resample 8 → 16 kHz	Wajib untuk wav2vec2 dan Whisper pre-trained
3 teknik augmentation (noise, stretch, pitch)	Mengatasi dataset kecil (394 train)
Augmentation hanya di train	Val/test tetap fair, tidak bias
Stratified by intent_class	Balance topik banking di semua partisi
Uppercase saat tokenize wav2vec2	Vocab CTC wav2vec2-base-960h hanya huruf kapital
Dua collator terpisah	wav2vec2 = CTC, Whisper = seq2seq
-100 untuk pad token label	Konvensi Hugging Face untuk ignore di loss

Kami memilih membandingkan **dua arsitektur ASR yang berbeda secara fundamental**: wav2vec2 (CTC-based) dan Whisper (encoder-decoder seq2seq), untuk melihat trade-off masing-masing pada domain banking telephony berbahasa Inggris.

Wav2Vec2 Architecture (CTC)



Whisper Architecture (Seq2Seq)



Pemilihan kedua model bertujuan membandingkan paradigma berbeda: pre-training homogen (wav2vec2) vs heterogen (Whisper), dan loss function CTC (frame-level) vs seq2seq (sequence-level).

wav2vec2 (CTC)

- Self-supervised pre-training di 960 jam **LibriSpeech** (audio studio English)
- **CTC loss** untuk alignment audio-text tanpa supervision frame-level
- Output character-level dengan vocab uppercase (32 token)
- Decoding: argmax per frame, collapse repeated, remove blank
- Cocok untuk task transkripsi pendek dengan format teks seragam

Whisper (Seq2Seq)

- Weakly-supervised training di **680 ribu jam audio multi-domain (web)**
- **Cross-entropy loss** dengan teacher forcing
- Output BPE subword tokens via tokenizer multilingual
- Decoding: autoregressive .generate() sampai end-of-sequence
- Robust ke variasi domain (telephony, podcast, meeting, dll)

*Tahap modelling dilakukan dalam **dua fase**: baseline evaluation untuk mengukur performa awal model pre-trained tanpa fine-tuning, lalu fine-tuning untuk adaptasi ke domain banking telephony. Kedua model dievaluasi dengan metrik standar ASR yaitu **WER (Word Error Rate)** dan **CER (Character Error Rate)** pada test set yang sama, agar perbandingan adil dan terukur.*

Sebelum fine-tuning, kami evaluasi kedua **model pre-trained tanpa training tambahan** di test set MINDS-14 untuk mengukur lower-bound performa awal. Skor baseline ini akan jadi referensi untuk mengukur seberapa besar improvement yang didapat setelah fine-tuning.

Inference Setup

- Device:** CUDA (NVIDIA RTX 4060 Laptop GPU, 8 GB)
- Library:** transformers, jiwer, torch
- Mode:** model.eval() + @torch.no_grad()
- Test set:** 85 sampel (15% dari dataset, stratified)

wav2vec2-base-960h

- Pre-training:** 960 jam LibriSpeech (studio English)
- Decoding:** Argmax per frame, collapse, remove blank
- Output:** Karakter uppercase, di lower-case di akhir
- Vocab:** 32 token CTC

whisper-base

- Pre-training:** 680 ribu jam audio multi-domain
- Decoding:** .generate() autoregressive
- Config:** language='en', task='transcribe'
- Output:** BPE subword token, di-lowercase

Karakteristik Audio

- Test audio:** WAV mono, sudah resample 16 kHz
- Domain:** banking telephony, English (en-US)
- Durasi:** variabel, 1,7s sampai 58s
- Tantangan utama:** domain mismatch dengan studio audio

Baseline evaluation memakai pre-trained checkpoint langsung tanpa training, hanya inferensi di test set. Hasil akan dilihat di slide Evaluation.

Pada tahap fine-tuning wav2vec2, **arsitektur model tetap sama dengan baseline** yaitu Wav2Vec2ForCTC dengan CTC loss. Yang berubah hanya bobot model yang akan beradaptasi ke domain banking telephony melalui training di train set 788 sampel (sudah termasuk augmentation).

Training Configuration

- **learning_rate** : $1e-4$ (3x lebih besar dari default)
- **num_train_epochs** : 20 (cukup untuk dataset kecil)
- **per_device_train_batch_size** : 4
- **gradient_accumulation_steps** : 2 (effective batch 8)
- **warmup_steps** : 200 (10% total steps)
- **weight_decay** : 0.005
- **fp16** : True (mixed precision)

Dampak Hyperparameter

- **Learning rate**: makin besar belajar lebih cepat tapi tidak stabil
- **Batch size**: kecil = lebih cepat tapi noisy gradient
- **Epochs**: makin banyak risiko overfitting di dataset kecil
- **Warmup**: cegah gradient meledak di awal training
- **Weight decay**: regularisasi L2 untuk cegah overfitting
- **fp16**: speedup 1,5 sampai 2 kali tanpa loss akurasi

Setup Khusus wav2vec2

- **freeze_feature_encoder()** : freeze CNN, hanya transformer di-train
- **Trainable**: 90,2M / 94,4M params (95,6%)
- **ctc_loss_reduction='mean'** : stabil untuk batch variabel
- **Uppercase fix**: label tokenisasi pakai .upper() karena vocab CTC

Best Practice Reference

- Mengikuti paper **Baevski et al., 2020** (wav2vec 2.0)
- Tutorial Hugging Face "Fine-Tune wav2vec2 for English ASR"
- LR range tutorial: $1e-5$ sampai $1e-4$ (kami pakai upper bound)
- Tidak dilakukan grid search formal karena keterbatasan compute
- Future work: ablation study dengan 3 sampai 4 nilai LR alternatif

Hyperparameter dipilih berdasarkan best practice paper wav2vec2 dan tutorial Hugging Face. Feature encoder di-freeze untuk hemat parameter dan cegah overfitting di dataset kecil.

*Fine-tuning Whisper memakai **WhisperForConditionalGeneration** dengan **seq2seq loss**, berbeda dari wav2vec2 yang CTC-based. Konsekuensinya, training memakai Seq2SeqTrainer dengan `predict_with_generate=True` agar evaluasi merefleksikan inference behavior yang sebenarnya (autoregressive generation).*

Training Configuration

- learning_rate** : 1e-5 (10x lebih kecil dari wav2vec2)
- num_train_epochs** : 10 (Whisper konvergen lebih cepat)
- per_device_train_batch_size** : 4
- gradient_accumulation_steps** : 4 (effective batch 16)
- warmup_steps** : 50
- weight_decay** : 0.01
- predict_with_generate** : True (wajib seq2seq)
- generation_max_length** : 225

Kenapa LR Lebih Kecil?

- Whisper sudah robust dari pre-training 680 ribu jam
- Hanya butuh **adjustment halus** ke domain baru
- LR besar (1e-4) bisa **merusak knowledge** yang sudah ada
- 1e-5 adalah default tutorial Hugging Face untuk Whisper
- Epoch lebih sedikit (10 vs 20) karena konvergensi cepat

Setup Khusus Whisper

- generation_config.language='en'**: skip language detection
- generation_config.task='transcribe'**: bukan translate
- forced_decoder_ids=None**: hapus API lama (deprecated)
- use_cache=False** saat **training**: cegah konflik dengan checkpointing
- All layers trainable**: 72,6M params (no freezing)

Best Practice Reference

- Mengikuti paper **Radford et al., 2022** (Whisper)
- Tutorial Hugging Face "Fine-Tune Whisper For Multilingual ASR"
- Config seq2seq berbeda dari CTC, butuh Seq2SeqTrainer
- `predict_with_generate=True` mahal compute tapi akurat
- Future work: coba whisper-small atau whisper-medium

Hyperparameter mengikuti tutorial resmi Hugging Face untuk Whisper. Learning rate 10x lebih kecil dari wav2vec2 karena Whisper sudah cukup robust dan hanya butuh adjustment halus.

Perbandingan Final – Baseline vs Fine-tuned:

	WER (%)	CER (%)
model		
wav2vec2-base-960h	64.60	46.42
wav2vec2-finetuned	42.28	33.60
whisper-base	52.85	41.16
whisper-finetuned	33.89	30.08

Insight:

1. Semua model mengalami peningkatan signifikan setelah fine-tuning
2. Whisper Fine Tuned performa terbaik, WER: 33,89 % (terendah), CER: 30,08 % (terendah)
3. Wave2Vec2, Baseline: WER **64.60%**, Fine-tuned: turun ke **42.28%** (improvement besar), masih kompetitif, tapi lebih sensitif terhadap noise
4. Whisper, Baseline: WER **52.85%**, Fine-tuned: turun ke **33.89%** (terbaik overall), baik dalam generalisasai audio dan robust terhadap variasi data
5. Fine tuning --> impact paling signifikan terhadap performa

Berdasarkan grafik terlihat model mengalami peningkatan setelah fine tuning. Whisper fine tuned menunjukkan performa terbaik dengan WER dan CER terendah, mengindikasikan kemampuan transkripsi yang lebih akurat dan robust dibandingkan model lainnya

wav2vec2

Baseline wav2vec2

```
-----
REF : the office not working correctly
PRED: the off oes not lookin collectly
---
REF : i would like to change my address
PRED: i would like to change my address
---
REF : how much money would i be able to withdraw the atm
PRED: how much money would i beb to withdraw at the atia
---
```

Contoh prediksi wav2vec2 fine-tuned:

```
Contoh prediksi wav2vec2 fine-tuned:
=====
REF : the office not working correctly
PRED : off is not working plosely
-----
REF : i would like to change my address
PRED : i would like to change my address
-----
REF : how much money would i be able to withdraw the atm
PRED : how much money would i is to withdraw at the atm
-----
REF : i'm making a large purchase and i'm trying to use the confirmation code that i received through text message
PRED : i'm making a large purchase and i'm trying to use the confirmation code that i was seazed through a text message
-----
REF : account balance please
PRED : account balance please
-----
```

Whisper

Baseline Whisper

```
REF : the office not working correctly
PRED: that office is not working correctly.
---
REF : i would like to change my address
PRED: i would like to change my address.
---
REF : how much money would i be able to withdraw the atm
PRED: how much money would i be if i withdraw at the atm?
---
```

Contoh prediksi Whisper

```
Contoh prediksi Whisper fine-tuned:
=====
REF : the office not working correctly
PRED : oh my office is not working properly
-----
REF : i would like to change my address
PRED : i would like to change my address
-----
REF : how much money would i be able to withdraw the atm
PRED : how much money would i pay to withdraw at the atm
-----
REF : i'm making a large purchase and i'm trying to use the confirmation code that i received through text message
PRED : i'm making a large purchase and i'm trying to use the confirmation code that i received through a text messag
e
-----
REF : account balance please
PRED : account balance please
-----
```



Kesimpulan

Pipeline ASR end-to-end berhasil dibangun dengan dua arsitektur model yang berbeda (wav2vec2 CTC dan Whisper seq2seq) di dataset banking telephony MINDS-14 (en-US, 563 sampel).

EDA mengungkap karakteristik kunci dataset:

- Audio telepon 8 kHz seragam, perlu resample ke 16 kHz
- Distribusi durasi right-skewed (median 6,4s, max 58s)
- 14 intent banking seimbang (rasio 1,41), total 1,34 jam
- Audio bersih (no clipping, no silent), tidak butuh filtering tambahan

Preprocessing pipeline mencakup 10 tahap utama:

- Normalisasi teks, resample, augmentation (3 teknik), stratified split
- Augmentation hanya di train set (394 ke 788 sampel)
- Dua processor terpisah (CTC + seq2seq) dengan uppercase fix khusus
- wav2vec2

Hasil baseline mengonfirmasi domain mismatch:

- wav2vec2-base-960h (LibriSpeech studio): WER 64,60%, CER 46,42%
- whisper-base (multi-domain 680k jam): WER 52,85%, CER 41,16%
- Whisper unggul 11,75 poin WER karena training data lebih beragam

Fine-tuning berhasil meningkatkan kedua model secara signifikan:

- wav2vec2 fine-tuned: WER 42,28%, CER 33,60% (improvement 22,32 poin WER)
- Whisper fine-tuned: WER 33,89%, CER 30,08% (improvement 18,96 poin WER)
- Whisper konsisten unggul di kedua kondisi (gap 8 sampai 12 poin WER)

Insight teknis penting:

- Tokenizer wav2vec2-base-960h hanya support uppercase, perlu .upper() sebelum tokenisasi label
- Whisper auto-add punctuation di baseline, hilang setelah fine-tuning ke format MINDS-14
- Gradient checkpointing Whisper bermasalah saat aktif bersama use_cache, dimatikan total untuk stabilitas
- Augmentation hanya berdampak setelah fine-tuning, tidak terlihat di baseline (test set tidak di-augment)

1.



Eksperimen Hyperparameter

- Grid Search / Bayesian Optimization (**Optuna**) → tuning *learning rate* & *epoch*
- Ablation Study → uji kombinasi augmentation (noise, time-stretch, dll)
- Regularisasi → eksplorasi **dropout & weight decay**

2.



Skala Model & Data

- Upgrade model → whisper-small, whisper-medium (trade-off WER vs compute)
- Penambahan data → **Common Voice, Switchboard**
- Multi-language → fine-tuning MINDS-14 (de-DE, fr-FR, es-ES, dll)

3.



Teknik Lanjutan

- **Language Model Rescoring (KenLM)** → potensi ↓ WER 5–10 poin
- **Beam Search Decoding** → lebih optimal dibanding greedy
- **SpecAugment** → augmentasi domain spektrogram
- **Mixup / CutMix Audio** → meningkatkan generalisasi

4.



Evaluasi Lebih Komprehensif

- Tambah metrik → **MER, BLEU**
- Error analysis → per intent & pola kesalahan
- **Confusion matrix kata** → identifikasi kesalahan umum
- **Latency benchmark** → kesiapan real-time

5.



Aplikasi Hilir

- Pipeline **ASR** → **NLP (intent classification)**
- End-to-end **audio** → **intent model**
- **Streaming inference** → real-time call center
- Optimasi deployment → **quantization (INT8), distillation**

6.



Production Readiness

- API service → **FastAPI / Gradio**
- Deployment → **Docker** (reproducible)
- **A/B Testing** → wav2vec2 vs Whisper
- Monitoring → drift detection & retraining

Dari sisi model, **optimasi hyperparameter, eksplorasi model yang lebih besar**, serta **penerapan teknik lanjutan** seperti beam search dan language model rescoring untuk menurunkan error rate. Dari sisi data dan evaluasi, akan **ditambahkan dataset eksternal, multi-language training, serta evaluasi yang lebih komprehensif termasuk error analysis dan latency benchmarking**. Selain itu, sistem akan diarahkan menuju **implementasi nyata melalui pengembangan pipeline ASR terintegrasi, deployment berbasis API dan Docker, serta A/B testing dan monitoring untuk memastikan performa tetap optimal**.

Thank You

Group C – Batch 10 NLP